Notation summary

Progressive draft

Generally, we use a consistent notation among the most important course parts. Only on the need and in order to simplify, slight *local* variations are made and specified according to the context (as indicated in the specific lectures).

The following guide shows the general case and can help to summarize the main variations according to the different sub-cases.

DATA:

Pattern	x_1	x_2	x_i	x_n
Pat *1*	$x_{1,1}$	$x_{1,2}$	…	$x_{1,n}$
…	…	…	…	…
Pat *p*	$x_{p,1}$	$x_{p,2}$	$x_{p,i}$	$x_{p,n}$
…				

$\boldsymbol{X}$ is a matrix $l \times n$

l rows (patterns),

n columns (features/variables/attributes/components)

$p=1..l, \quad i=1..n$

We often need to omit some indices when the context is clear, e.g.:

- Each row, generic $\boldsymbol{x}$ (vector-bold): a raw in the table: (input) example, pattern, instance, sample, input vector, ….
- x_i or x_j (scalar): component i or j (given a pattern $\boldsymbol{x}$, i.e. omitting p)
- $\boldsymbol{x}_p$ (or rarely other indices e.g. $\boldsymbol{x}_i$) (vector-bold): p-*th* (i-*th*) raw in the table = pattern p (or i)
- $x_{p,i}$ (scalar) also as $(\boldsymbol{x}_p)_i$: component i of the pattern p
  or we can often use also $\boldsymbol{x}_{p,j}$ for the component j, *etc.*
- For the target we will typically use just y_p with $p=1..l$ (the same for d_p or t_p)

General:

- $\boldsymbol{x}$ is the input pattern
- o (or sometimes *out)* is the model output component (e.g. an the output of a unit in Neural Networks)
- $h(\boldsymbol{x})$ is the hypothesis/model output: the output of the model $h(\boldsymbol{x})$ can be of different nature according to the task (e.g. discrete for classification, a real number for regression).

- h is the hypothesis emitted by the learning algorithm to approximate the unknow target function f.
- $h_{\boldsymbol{w}}$ can be used to highlight the dependencies on the parameters $\boldsymbol{w}$.

Error and Risk: expected value of the "inner" loss/measure $L(h(\boldsymbol{x}),d)$ that changes according to the tasks and models (see first- intro lectures and validation lectures).

- Typically $E(\boldsymbol{w})=\frac{1}{r}\sum_{p=1}^{r} L(h(\boldsymbol{x}_p), d_p)$ over a generic set of r data (it can be measured for training, validation or test according to the context)
- A typical example is the MSE used for the Linear models and Neural Networks, for l training data:

$$E(\boldsymbol{w}) = \frac{1}{l}\sum_{p=1}^{l}(d_p - h_{\boldsymbol{w}}(\boldsymbol{x}_p))^2$$

- Risk function over all the data: R, see SLT and Validation lectures
- Empirical risk: R_{emp} = $E(\boldsymbol{w})$ error, surrogate of R on a finite set of data, typically used for the training data (see SLT, whereas $R_{emp} = R_{emp}(h, TR)$) or also for a validation/test part (see Validation lectures, specifying the h and the data split D in the form $R_{emp}(h, D)$, within the purpose of training, e.g. on D_{TR} in the simple hold-out, model selection, e.g. on D_{VL} in the simple hold-out, or model assessment, e.g. on D_{TS} in the simple hold-out, as specified in the model selection/assessment algorithms).

Objective function for model training: it can be of different nature according to the task and model (also changing the "inner" loss L) and it can (or not) include regularization aspects, e.g. for Linear models and Neural Networks with Tikhonov regularization the objective function is the following *Loss* function (see the lectures for details):

$$Loss(\boldsymbol{w}) (\text{or } Loss(h_{\boldsymbol{w}})) = \text{ error data term (MSE) + penalty term}$$

Specific cases

In Linear models/LTU (the basic):

As in the table above:

- Inputs: $\boldsymbol{x}$
- Outputs: $h(\boldsymbol{x})$
- Targets: typically y (regression); y or d (classification)

In Neural Networks:

- Inputs: $\boldsymbol{x}$ (loaded in the input layer). Also, $\boldsymbol{x}$ can represent a generic input to each unit, either from extern source (input pattern) or another (also hidden) unit according to the position of the unit in the network.
  As explained in the slide "The NN and their Units" with a picture: If we load the pattern $\boldsymbol{x}$ in the input layer, we can use the notation with $\boldsymbol{o}$ for both the inputs and the hidden units outputs. Hence, *inside the network*, the input to each unit t (either hidden and output

units) from *any source u* (through the connection w_{tu}) is typically denoted as o_u (this is the style used in the Backpropagation derivation).

- Outputs: $h(\boldsymbol{x}) = \boldsymbol{o}(\boldsymbol{x})$ at last layer (with more than one output units) and $h(\boldsymbol{x}_p) = \boldsymbol{o}(\boldsymbol{x}_p)$.
  In Backprop derivation we simplified using o_k (or o_u etc.) as output of the unit k (or u) for a pattern p, i.e. omitting p (and the same for other vectors used therein for a single pattern p).
  I.e. the subscript is used therein for the indices of the units (see also such usage explained above after the main data table for components of a vector omitting p) .
- Targets: d scalar (or vector $\boldsymbol{d}$ for multioutput networks)

In SVM:

- Inputs: **x**
- Output: $h(\mathbf{x})$
- Targets: d (as for classification above)

In SVM the bias w_0 is typically denoted as b

Slight variations to be consistent with Haykin chapter 6:
$l \rightarrow$ N (number of data), $n \rightarrow$ m (input dimension)
We use i and j for the patterns indices.

Special cases (different domains)

In the Introduction for Concept Learning (Inductive Bias) part

This is a particular case due to the discrete nature of the hypothesis space in propositional logic (and we partially use the nice framework from the Mitchel presentation, with very slight changes w.r.t. to the previous style)

Target function: d: X→{0,1} (c: X→{0,1} in the Mitchel book)
TR set couples <$\boldsymbol{x}$, $d(\boldsymbol{x})$> in TR (or D in the Mitchel book)
h: X→{0,1} (made with literals l_i of propositional logics)

- Inputs: $\boldsymbol{x}$
- Outputs: $h(\mathbf{x})$
- Targets: $d(\boldsymbol{x})$

In RNN:

The domain changes from vectors to sequences/time-series:

- Inputs: $\boldsymbol{l}(t)$, components of a sequence
- Outputs: $o(t)$ or $y(t)$
- Targets: $d(t)$
- $\boldsymbol{x}(t)$ in RNN denotes the states values